Textual Analysis in Finance

Week 3: Neural Embedding

Tri, Minh Phan

Faculty of Business and Economics
University of Basel

Where are we?

↑ **We are here**

Outline

Word2Vec

Issues with word-count methods

Bag-of-Words (or *tf.idf*)

- ... treats words as **independent units**
  - "profit" $\neq$ "income"
  - "bullish" $\neq$ "bearish"

Issues with word-count methods

Bag-of-Words (or *tf.idf*)

- ... treats words as **independent units**
  - "profit" $\neq$ "income"
  - "bullish" $\neq$ "bearish"
- ... produces **high-dimensional** and **sparse** features
  - The document-term (or *tf.idf*) matrix has $|V|$ columns and a lot of zeros

Issues with word-count methods

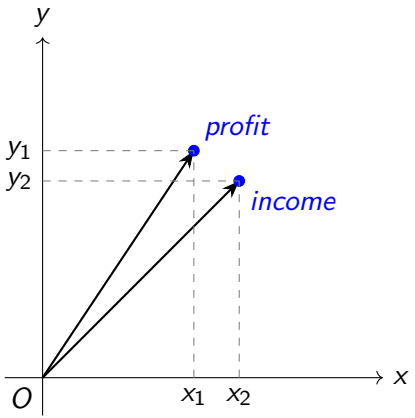
Bag-of-Words (or *tf.idf*)

- ... treats words as **independent units**
  - "profit" $\neq$ "income"
  - "bullish" $\neq$ "bearish"
- ... produces **high-dimensional** and **sparse** features
  - The document-term (or *tf.idf*) matrix has $|V|$ columns and a lot of zeros

Neural embedding approaches overcome those limitations!!!

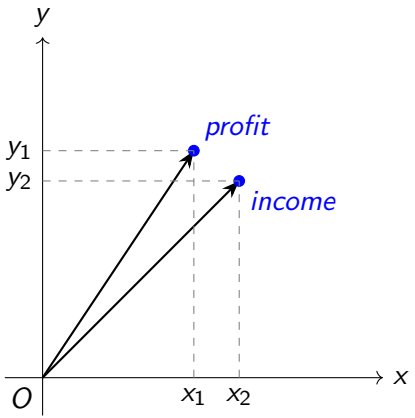
The core idea - Embedding

- An **embedding** is a learned mapping from discrete objects (e.g., words or documents) into a **low-dimensional continuous** vector space.



The core idea - Embedding

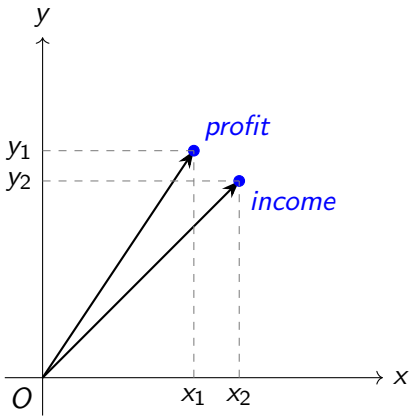
- An **embedding** is a learned mapping from discrete objects (e.g., words or documents) into a **low-dimensional continuous** vector space.



- "profit" $\rightarrow (x_1, y_1)$; "income" $\rightarrow (x_2, y_2)$

The core idea - Embedding

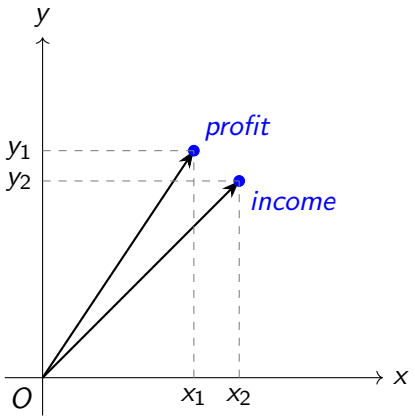
- An **embedding** is a learned mapping from discrete objects (e.g., words or documents) into a **low-dimensional continuous** vector space.



- "profit" $\rightarrow (x_1, y_1)$; "income" $\rightarrow (x_2, y_2)$
- (x_1, y_1) is the **embedding** of "profit"

The core idea - Embedding

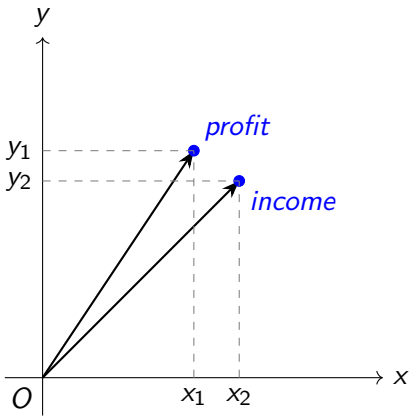
- An **embedding** is a learned mapping from discrete objects (e.g., words or documents) into a **low-dimensional continuous** vector space.



- "profit" $\rightarrow (x_1, y_1)$; "income" $\rightarrow (x_2, y_2)$
- (x_1, y_1) is the **embedding** of "profit"
- How to find x_1, y_1 and x_2, y_2?

The core idea - Embedding

- An **embedding** is a learned mapping from discrete objects (e.g., words or documents) into a **low-dimensional continuous** vector space.



- "profit" $\rightarrow (x_1, y_1)$; "income" $\rightarrow (x_2, y_2)$
- (x_1, y_1) is the **embedding** of "profit"
- How to find x_1, y_1 and x_2, y_2?
- We find the embeddings such that ***words that appear in similar contexts should have similar embeddings***

Continuous Bag-of-Words (CBOW) [Mikolov et al., 2013a] - General idea

- Consider the sentence:

  The company reported high earnings last quarter

Continuous Bag-of-Words (CBOW) [Mikolov et al., 2013a] - General idea

- Consider the sentence:

  The company reported high earnings last quarter

- In **CBOW**, the task is: given the **context**, predict the **target word**. For example:

  The company reported high $\langle$ *?* $\rangle$ *last quarter*

Continuous Bag-of-Words (CBOW) [Mikolov et al., 2013a] - General idea

- Consider the sentence:

  The company reported high earnings last quarter

- In **CBOW**, the task is: given the **context**, predict the **target word**. For example:

  The company reported high $\langle$ *?* $\rangle$ *last quarter*

- The **context** is defined as the set of words surrounding the target word within a pre-defined **window size**. Example: window $= 2 \rightarrow$ context $=$ {"reported", "high", "last", "quarter"}

Continuous Bag-of-Words (CBOW) [Mikolov et al., 2013a] - General idea

- Consider the sentence:

  The company reported high earnings last quarter

- In **CBOW**, the task is: given the **context**, predict the **target word**. For example:

  The company reported high $\langle$ *?* $\rangle$ *last quarter*

- The **context** is defined as the set of words surrounding the target word within a pre-defined **window size**. Example: window $= 2 \rightarrow$ context $=$ {"reported", "high", "last", "quarter"}
- CBOW predicts the target word using a **one-hidden-layer neural network** with an **identity activation** $\rightarrow$ the "neural" in neural embedding comes from this structure

Continuous Bag-of-Words (CBOW) [Mikolov et al., 2013a] - General idea

- Consider the sentence:

  The company reported high earnings last quarter

- In **CBOW**, the task is: given the **context**, predict the **target word**. For example:

  The company reported high $\langle$ *?* $\rangle$ *last quarter*

- The **context** is defined as the set of words surrounding the target word within a pre-defined **window size**. Example: window $= 2 \rightarrow$ context $=$ {"reported", "high", "last", "quarter"}
- CBOW predicts the target word using a **one-hidden-layer neural network** with an **identity activation** $\rightarrow$ the "neural" in neural embedding comes from this structure
- The **embeddings** are the weights from the input layer to the hidden layer

Continuous Bag-of-Words - How to?

- **Step 1:** Define the vocabulary - {“company”, “earnings”, “high”, “last”, “quarter”, “reported”}; the vocabulary size $|V| = 6$

Continuous Bag-of-Words - How to?

- **Step 1:** Define the vocabulary - {"company", "earnings", "high", "last", "quarter", "reported"}; the vocabulary size $|V| = 6$
- **Step 2:** One-hot encoding - Each word is represented as a binary vector of length $|V|$, with a value of **1 at the index corresponding to the word in the vocabulary and 0 elsewhere**
  - "reported" $\rightarrow x_{reported} = [0, 0, 0, 0, 0, 1]$
  - "high" $\rightarrow x_{high} = [0, 0, 1, 0, 0, 0]$
  - "last" $\rightarrow x_{last} = [0, 0, 0, 1, 0, 0]$
  - "quarter" $\rightarrow x_{quarter} = [0, 0, 0, 0, 1, 0]$
  - "earnings" $\rightarrow y = x_{earnings} = [0, 1, 0, 0, 0, 0]$

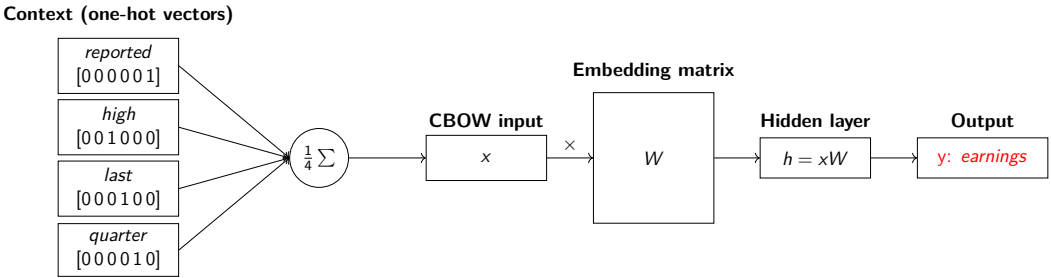
Continuous Bag-of-Words - How to?

- **Step 1:** Define the vocabulary - {"company", "earnings", "high", "last", "quarter", "reported"}; the vocabulary size $|V| = 6$
- **Step 2:** One-hot encoding - Each word is represented as a binary vector of length $|V|$, with a value of **1 at the index corresponding to the word in the vocabulary and 0 elsewhere**
  - "reported" $\rightarrow x_{reported} = [0, 0, 0, 0, 0, 1]$
  - "high" $\rightarrow x_{high} = [0, 0, 1, 0, 0, 0]$
  - "last" $\rightarrow x_{last} = [0, 0, 0, 1, 0, 0]$
  - "quarter" $\rightarrow x_{quarter} = [0, 0, 0, 0, 1, 0]$
  - "earnings" $\rightarrow y = x_{earnings} = [0, 1, 0, 0, 0, 0]$
- **Step 3:** CBOW multiplies the one-hot encoding of each context word by the embedding matrix W, i.e., $x_{reported}W$

Continuous Bag-of-Words - How to?

- **Step 1:** Define the vocabulary - {"company", "earnings", "high", "last", "quarter", "reported"}; the vocabulary size $|V| = 6$
- **Step 2:** One-hot encoding - Each word is represented as a binary vector of length $|V|$, with a value of **1 at the index corresponding to the word in the vocabulary and 0 elsewhere**
  - "reported" $\rightarrow x_{reported} = [0, 0, 0, 0, 0, 1]$
  - "high" $\rightarrow x_{high} = [0, 0, 1, 0, 0, 0]$
  - "last" $\rightarrow x_{last} = [0, 0, 0, 1, 0, 0]$
  - "quarter" $\rightarrow x_{quarter} = [0, 0, 0, 0, 1, 0]$
  - "earnings" $\rightarrow y = x_{earnings} = [0, 1, 0, 0, 0, 0]$
- **Step 3:** CBOW multiplies the one-hot encoding of each context word by the embedding matrix W, i.e., $x_{reported}W$
- **Step 4:** The CBOW input is the average of the context word vectors, $x = \frac{1}{4}(x_{reported} + x_{high} + x_{last} + x_{quarter}) = [0, 0, 0.25, 0.25, 0.25, 0.25]$, x is a $1 \times |V|$ vector

Continuous Bag-of-Words - How to?



Continuous Bag-of-Words - Training

- **Constructing the CBOW training data.** Start from the text:

  The company reported high earnings last quarter

  - Preprocess the text (lowercasing, lemmatization): [“company”, “report”, “high”, “earning”, “last”, “quarter”]
  - Fix a context window size (e.g., window = 2).
  - Slide the window across the text to generate training examples:
    - Target: “company” - Context: {“report”, “high”} - **Edge case, can be ignored**
    - Target: “report” - Context: {“company”, “high”, “earning”}
    - Target: “high” - Context: {“company”, “report”, “earning”, “last”}
    - Target: “earning” - Context: {“report”, “high”, “last”, “quarter”}
    - Target: “last” - Context: {“high”, “earning”, “quarter”}
    - Target: “quarter” - Context: {“earning”, “last”} - **Edge case, can be ignored**

Continuous Bag-of-Words - Training

- **Constructing the CBOW training data.** Start from the text:

  The company reported high earnings last quarter

  - Preprocess the text (lowercasing, lemmatization): ["company", "report", "high", "earning", "last", "quarter"]
  - Fix a context window size (e.g., window = 2).
  - Slide the window across the text to generate training examples:
    - Target: "company" - Context: {"report", "high"} - **Edge case, can be ignored**
    - Target: "report" - Context: {"company", "high", "earning"}
    - Target: "high" - Context: {"company", "report", "earning", "last"}
    - Target: "earning" - Context: {"report", "high", "last", "quarter"}
    - Target: "last" - Context: {"high", "earning", "quarter"}
    - Target: "quarter" - Context: {"earning", "last"} - **Edge case, can be ignored**

- The predicted output: $\hat{y}_j = \frac{\exp(h\tilde{w}_j^\top)}{\sum_{i=1}^{|V|}\exp(h\tilde{w}_i^\top)}$ ($y = [y_1, ..., y_{|V|}]$; $h = xW$; $\tilde{w}_j$ is a row of $\tilde{W}$) is very costly to compute because of the denominator

Continuous Bag-of-Words - Training

- Intuitively, predicting y means **selecting the word in the vocabulary that best fits the given context** $\rightarrow$ when the vocabulary is large, selecting takes time!

Continuous Bag-of-Words - Training

- Intuitively, predicting y means **selecting the word in the vocabulary that best fits the given context** $\rightarrow$ when the vocabulary is large, selecting takes time!
- Solution? **Negative sampling** [Mikolov et al., 2013b]!
  - Normal CBOW: given a context, predict the correct word among all words in the vocabulary.
  - Negative sampling: given a context and a candidate word, ask the model: **is this the correct word?**
    $\rightarrow$ It turns a multi-class classification into a **binary classification** (yes/no)!

Continuous Bag-of-Words - Training

- Intuitively, predicting y means **selecting the word in the vocabulary that best fits the given context** $\rightarrow$ when the vocabulary is large, selecting takes time!
- Solution? **Negative sampling** [Mikolov et al., 2013b]!
  - Normal CBOW: given a context, predict the correct word among all words in the vocabulary.
  - Negative sampling: given a context and a candidate word, ask the model: **is this the correct word?**
    $\rightarrow$ It turns a multi-class classification into a **binary classification** (yes/no)!
- Example training pairs with negative sampling (context = {"report", "high", "last", "quarter"}):
  - Word: "earning" - Target: 1
  - Word: "company" - Target: 0
  - Word: "loss" - Target: 0
  - Word: "ceo" - Target: 0

  The last 3 examples are called **negative samples** and are randomly chosen!

Continuous Bag-of-Words - Remarks

- Choose the embedding dimension D beforehand; a common choice is $D = 300$

Continuous Bag-of-Words - Remarks

- Choose the embedding dimension D beforehand; a common choice is $D = 300$
- Choose the number of negative samples k:
  - Small datasets (<10M words): $k = 5$–20
  - Large datasets (>1B words): $k = 2$–5 [Mikolov et al., 2013b]

Continuous Bag-of-Words - Remarks

- Choose the embedding dimension D beforehand; a common choice is $D = 300$
- Choose the number of negative samples k:
  - Small datasets (<10M words): $k = 5$–20
  - Large datasets (>1B words): $k = 2$–5 [Mikolov et al., 2013b]
- Negative sampling not only reduces training cost but also often improves the **quality of the learned embeddings**

Continuous Bag-of-Words - Remarks

- Choose the embedding dimension D beforehand; a common choice is $D = 300$
- Choose the number of negative samples k:
  - Small datasets (<10M words): $k = 5$–20
  - Large datasets (>1B words): $k = 2$–5 [Mikolov et al., 2013b]
- Negative sampling not only reduces training cost but also often improves the **quality of the learned embeddings**
- Besides CBOW, there are some other variants of Word2Vec:
  - Skip-gram [Mikolov et al., 2013a]: given a word, predict the context
  - **GloVe** [Pennington et al., 2014]: learns embeddings from a global **co-occurrence matrix**. If words i and j appear together X_{ij} times in the corpus, their embeddings satisfy: $w_i^\top w_j + b_i + b_j \approx \log(X_{ij})$
  - Levy and Goldberg [2014] show that CBOW is equivalent to **factorizing a co-occurrence matrix via SVD**

Continuous Bag-of-Words - Remarks

- Choose the embedding dimension D beforehand; a common choice is $D = 300$
- Choose the number of negative samples k:
  - Small datasets (<10M words): $k = 5$–20
  - Large datasets (>1B words): $k = 2$–5 [Mikolov et al., 2013b]
- Negative sampling not only reduces training cost but also often improves the **quality of the learned embeddings**
- Besides CBOW, there are some other variants of Word2Vec:
  - Skip-gram [Mikolov et al., 2013a]: given a word, predict the context
  - **GloVe** [Pennington et al., 2014]: learns embeddings from a global **co-occurrence matrix**. If words i and j appear together X_{ij} times in the corpus, their embeddings satisfy: $w_i^\top w_j + b_i + b_j \approx \log(X_{ij})$
  - Levy and Goldberg [2014] show that CBOW is equivalent to **factorizing a co-occurrence matrix via SVD**
- There is Doc2Vec [Le and Mikolov, 2014], which is similar to Word2Vec but for documents (not words)

Neural embedding in finance

- (Personal view) Economists jump from Bag-of-Words directly to large language models, bypassing neural embedding approaches like Word2Vec or Doc2Vec

Neural embedding in finance

- (Personal view) Economists jump from Bag-of-Words directly to large language models, bypassing neural embedding approaches like Word2Vec or Doc2Vec
- Word2Vec converts words (and documents) into dense vectors, enabling a variety of financial applications:
  - Measure corporate culture from earnings call transcripts [Li et al., 2021] (customized kNN)
  - Identify forward-looking statements in the Management's Discussion and Analysis section of 10-K filings [Brown et al., 2024] (supervised deep learning)
  - Classify business types using Item 1 in 10-K filings [Song, 2021] (kNN)

Neural embedding in finance

- (Personal view) Economists jump from Bag-of-Words directly to large language models, bypassing neural embedding approaches like Word2Vec or Doc2Vec
- Word2Vec converts words (and documents) into dense vectors, enabling a variety of financial applications:
  - Measure corporate culture from earnings call transcripts [Li et al., 2021] (customized kNN)
  - Identify forward-looking statements in the Management's Discussion and Analysis section of 10-K filings [Brown et al., 2024] (supervised deep learning)
  - Classify business types using Item 1 in 10-K filings [Song, 2021] (kNN)
- A key challenge of Word2Vec: it relies on labeled data for supervised tasks, which can be limited, making some supervised approaches difficult. E.g., Brown et al. [2024] have to annotate the text themselves

Neural embedding - Pros & Cons

- Pros:
  - Capture semantic meaning: words with similar meanings (e.g., “profit” and “income”) have nearby vector representations
  - Transform sparse, high-dimensional representations (e.g., document–term matrices) into dense, low-dimensional vectors
  - Enable downstream tasks (e.g., classification, clustering, similarity) that typically outperform Bag-of-Words–based approaches

Neural embedding - Pros & Cons

- Pros:
  - Capture semantic meaning: words with similar meanings (e.g., "profit" and "income") have nearby vector representations
  - Transform sparse, high-dimensional representations (e.g., document–term matrices) into dense, low-dimensional vectors
  - Enable downstream tasks (e.g., classification, clustering, similarity) that typically outperform Bag-of-Words–based approaches
- Cons:
  - **Static embeddings** - Once training is complete, each word is mapped to a single vector
    - Context is ignored - "bank" in "commercial bank" and "river bank" share the same vector
    - Poor handling of polysemy; e.g., "charge" can mean a fee or an accusation
  - Limited interpretability - Individual embedding dimensions have no clear semantic or economic meaning
  - Data-hungry - Large corpora are required to learn high-quality embeddings

Contextualized embeddings

Embeddings for Language Model - ELMo [Peters et al., 2018]

Limitations of static neural embeddings (e.g., Word2Vec, GloVe, Doc2Vec):

- They may produce identical (or very similar) sentence representations for:
  - *"revenue increased and cost decreased"* and
  - *"revenue decreased and cost increased"*,

  when sentence embeddings are constructed using simple averaging

Embeddings for Language Model - ELMo [Peters et al., 2018]

Limitations of static neural embeddings (e.g., Word2Vec, GloVe, Doc2Vec):

- They may produce identical (or very similar) sentence representations for:
  - *"revenue increased and cost decreased"* and
  - *"revenue decreased and cost increased"*,

  when sentence embeddings are constructed using simple averaging
- Each word has a single fixed vector
  - "bank" in *"commercial bank"* and "bank" in *"river bank"* share the same embedding

Embeddings for Language Model - ELMo [Peters et al., 2018]

Limitations of static neural embeddings (e.g., Word2Vec, GloVe, Doc2Vec):

- They may produce identical (or very similar) sentence representations for:
  - *"revenue increased and cost decreased"* and
  - *"revenue decreased and cost increased"*,

  when sentence embeddings are constructed using simple averaging
- Each word has a single fixed vector
  - "bank" in *"commercial bank"* and "bank" in *"river bank"* share the same embedding
- **ELMo's key idea**: generate **dynamic** word embeddings that depend on surrounding words and word order

Embeddings for Language Model - ELMo [Peters et al., 2018]

Limitations of static neural embeddings (e.g., Word2Vec, GloVe, Doc2Vec):

- They may produce identical (or very similar) sentence representations for:
  - *"revenue increased and cost decreased"* and
  - *"revenue decreased and cost increased"*,

  when sentence embeddings are constructed using simple averaging
- Each word has a single fixed vector
  - "bank" in *"commercial bank"* and "bank" in *"river bank"* share the same embedding
- **ELMo's key idea**: generate **dynamic** word embeddings that depend on surrounding words and word order
- As a result, ELMo produces different embeddings for "bank" in "commercial bank" and "bank" in "river bank"

Embeddings for Language Model - ELMo [Peters et al., 2018]

Limitations of static neural embeddings (e.g., Word2Vec, GloVe, Doc2Vec):

- They may produce identical (or very similar) sentence representations for:
  - *"revenue increased and cost decreased"* and
  - *"revenue decreased and cost increased"*,

  when sentence embeddings are constructed using simple averaging
- Each word has a single fixed vector
  - "bank" in *"commercial bank"* and "bank" in *"river bank"* share the same embedding
- **ELMo's key idea**: generate **dynamic** word embeddings that depend on surrounding words and word order
- As a result, ELMo produces different embeddings for "bank" in "commercial bank" and "bank" in "river bank"
- Intuitively: **ELMo = word embeddings + a sequential model**

ELMo's idea

- ELMo treats a sentence as a **time series of words** (w_t):
  - ["revenue", "increased", "and", "cost", "decreased"] $\rightarrow [w_1, w_2, w_3, w_4, w_5]$

ELMo's idea

- ELMo treats a sentence as a **time series of words** (w_t):
  - ["revenue", "increased", "and", "cost", "decreased"] $\rightarrow [w_1, w_2, w_3, w_4, w_5]$
- Each word has a pre-trained embedding (e.g., Word2Vec) x_t, $t = 1, \ldots, 5$
  - x_t captures the **general semantics** of w_t
  - But x_t is **context-independent**, i.e., it does not consider surrounding words

ELMo's idea

- ELMo treats a sentence as a **time series of words** (w_t):
  - ["revenue", "increased", "and", "cost", "decreased"] $\rightarrow [w_1, w_2, w_3, w_4, w_5]$
- Each word has a pre-trained embedding (e.g., Word2Vec) x_t, $t = 1, \ldots, 5$
  - x_t captures the **general semantics** of w_t
  - But x_t is **context-independent**, i.e., it does not consider surrounding words
- ELMo computes a **contextual embedding** e_t for each word w_t:

$$e_t = f(x_1, \ldots, x_5) \quad \text{or equivalently} \quad e_t = f(x_t, h_t),$$

where $h_t = g(x_1, \ldots, x_{t-1}, x_{t+1}, \ldots, x_5)$ summarizes the surrounding context

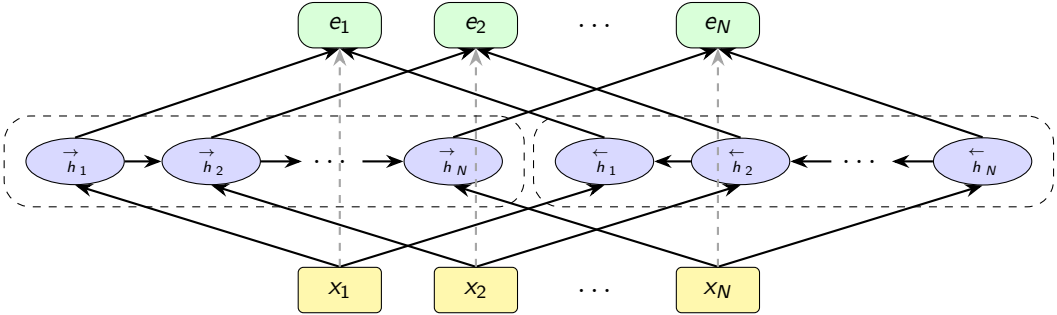
ELMo's idea

- ELMo treats a sentence as a **time series of words** (w_t):
  - ["revenue", "increased", "and", "cost", "decreased"] $\rightarrow [w_1, w_2, w_3, w_4, w_5]$
- Each word has a pre-trained embedding (e.g., Word2Vec) x_t, $t = 1, \ldots, 5$
  - x_t captures the **general semantics** of w_t
  - But x_t is **context-independent**, i.e., it does not consider surrounding words
- ELMo computes a **contextual embedding** e_t for each word w_t:

$$e_t = f(x_1, \ldots, x_5) \quad \text{or equivalently} \quad e_t = f(x_t, h_t),$$

  where $h_t = g(x_1, \ldots, x_{t-1}, x_{t+1}, \ldots, x_5)$ summarizes the surrounding context
- Notice that, $f(x_1, \ldots, x_5)$ also captures the word positions, meaning $f(x_1, x_2, x_3, x_4, x_5) \neq f(x_2, x_1, x_4, x_5, x_3)$

Embeddings for Language Model - ELMo



- $h_t = (\overrightarrow{h}_t^\top, \overleftarrow{h}_t^\top)^\top$, where $\overrightarrow{h}_t = \text{LSTM}(x_t, \overrightarrow{h}_{t-1})$ and $\overleftarrow{h}_t = \text{LSTM}(x_t, \overleftarrow{h}_{t+1})$

  $\rightarrow$ The embedding of a word depends on the words **before** and **after** it in the text!

- E.g., $\overrightarrow{h}_2 = g(x_2, \overrightarrow{h}_1) = g(x_2, g(x_1, \overrightarrow{h}_0))$; similarly $\overleftarrow{h}_{N-1} = g(x_{N-1}, g(x_N, \overleftarrow{h}_{N+1}))$; $\overrightarrow{h}_0, \overleftarrow{h}_N = (0, 0, ..., 0)$

ELMo - Remarks

- ELMo uses **bidirectional Long Short-Term Memory (BiLSTM)** networks [Hochreiter and Schmidhuber, 1997] as its sequential model
- The contextual embedding e_t for word w_t is typically a concatenation of:

$$e_t = \begin{bmatrix} x_t \\ \overrightarrow{h}_t \\ \overleftarrow{h}_t \end{bmatrix} = (x_t^\top, \overrightarrow{h}_t^\top, \overleftarrow{h}_t^\top)^\top$$

- **Context awareness**
  - $\overrightarrow{h}_t$ depends on all previous words $x_1, ..., x_{t-1}$ and hidden states $\overrightarrow{h}_1, ..., \overrightarrow{h}_{t-1}$
  - $\overleftarrow{h}_t$ depends on all future words $x_{t+1}, ..., x_T$ and hidden states $\overleftarrow{h}_{t+1}, ..., \overleftarrow{h}_T$
- The final embedding e_t is therefore **contextual**: the same word w_t gets different embeddings depending on its surrounding words

ELMo - Pros & Cons

- Pros:
  - **Contextualized embeddings**: ELMo addresses key limitations of Bag-of-Words and static embeddings (e.g., Word2Vec) by allowing word representations to vary with context
  - **Improved downstream performance**: consistently outperforms Bag-of-Words and static embeddings in tasks such as sentiment analysis, semantic similarity, and classification

ELMo - Pros & Cons

- Pros:
  - **Contextualized embeddings**: ELMo addresses key limitations of Bag-of-Words and static embeddings (e.g., Word2Vec) by allowing word representations to vary with context
  - **Improved downstream performance**: consistently outperforms Bag-of-Words and static embeddings in tasks such as sentiment analysis, semantic similarity, and classification
- Cons:
  - **Main limitation: reliance on sequential models (LSTMs)**
    - **Computationally expensive** due to sequential processing
    - Difficulty handling very long documents due to **vanishing gradients**
  - **Nearly no applications in finance** - less interpretable than BoW, but less powerful than language models

ELMo is computationally expensive

- Consider a document with T words: $[w_1, w_2, \ldots, w_T]$. The contextualized embedding of the last word w_T is

$$e_T = (x_t^\top, \overrightarrow{h}_t^\top, \overleftarrow{h}_t^\top)^\top$$

ELMo is computationally expensive

- Consider a document with T words: $[w_1, w_2, \ldots, w_T]$. The contextualized embedding of the last word w_T is

$$e_T = (x_t^\top, \overrightarrow{h}_t^\top, \overleftarrow{h}_t^\top)^\top$$

- Due to the sequential nature of the forward LSTM,

$$\overrightarrow{h}_t = g\left(\overrightarrow{h}_{t-1}, x_t\right),$$

which implies

$$\overrightarrow{h}_T = g\left(g\left(\cdots g\left(\overrightarrow{h}_0, x_1\right)\cdots, x_{T-1}\right), x_T\right)$$

ELMo is computationally expensive

- Consider a document with T words: $[w_1, w_2, \ldots, w_T]$. The contextualized embedding of the last word w_T is

$$e_T = (x_t^\top, \overrightarrow{h}_t^\top, \overleftarrow{h}_t^\top)^\top$$

- Due to the sequential nature of the forward LSTM,

$$\overrightarrow{h}_t = g\left(\overrightarrow{h}_{t-1}, x_t\right),$$

which implies

$$\overrightarrow{h}_T = g\left(g\left(\cdots g\left(\overrightarrow{h}_0, x_1\right)\cdots, x_{T-1}\right), x_T\right)$$

- $\overrightarrow{h}_T$ **cannot be computed in parallel**, so as T grows large:
  - The model must compute $\overrightarrow{h}_1, \overrightarrow{h}_2, \ldots, \overrightarrow{h}_{T-1}$ sequentially
  - This leads to high computational cost and slow training/inference for long documents

ELMo suffers from the vanishing-gradient problem

A neural network is trained by **Stochastic Gradient Descent**, meaning they update the network weights, h, by iteratively subtracting them from the gradients

$$h_t^{(n)} = h_t^{(n-1)} - \eta \frac{\partial L}{\partial h_t}\bigg|_{h_t^{(n-1)}}$$

then, by the chain rule

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}},$$

because, $h_k = g(Wh_{k-1} + b) \rightarrow \frac{\partial h_k}{\partial h_{k-1}} = Wg'(Wh_{k-1} + b)$

$$\left\| \frac{\partial h_k}{\partial h_{k-1}} \right\| \lesssim \|Wg'(.)\|$$

ELMo suffers from the vanishing-gradient problem

Therefore,

$$\left\| \frac{\partial L}{\partial h_t} \right\| \lesssim \|Wg'(.)\|^{T-t}$$

Usually in training, $\|Wg'(.)\| \leq 1$, so $\|Wg'(.)\|^{T-t} \to 0$ as T goes large, or it means

$$\left\| \frac{\partial L}{\partial h_t} \right\| \to 0, \quad T \to \infty$$

- In practice, during SGD, the embeddings of early tokens barely update, i.e., $h_1^{(n)} \approx h_1^{(n-1)}$ across many iterations
- Consequently, ELMo "forgets" ...
  - ... to update the embeddings of early words; and
  - ... long-range dependencies, treating **early words as having almost no influence on the final words** for long documents

Language models solve these problems!

Coming next

Coming next

- Hands-on practice
  - Exploring the pre-trained Google Word2Vec model
  - Training Word2Vec and Doc2Vec models using `gensim` and Tesla's annual statements
  - Notebook/Solution
- Next lecture
  - Transformers and Language Models

References I

Stephen V Brown, Lisa A Hinson, and Jennifer Wu Tucker. Financial statement adequacy and firms' MD&A disclosures. *Contemporary Accounting Research*, 41(1):126–162, 2024.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

Quoc Le and Tomas Mikolov. Distributed representations of sentences and documents. In *International Conference on Machine Learning*, pages 1188–1196. PMLR, 2014.

Omer Levy and Yoav Goldberg. Neural word embedding as implicit matrix factorization. *Advances in Neural Information Processing Systems*, 27, 2014.

Kai Li, Feng Mai, Rui Shen, and Xinyan Yan. Measuring corporate culture using machine learning. *The Review of Financial Studies*, 34(7): 3265–3315, 2021.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013a.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 2013b.

Jeffrey Pennington, Richard Socher, and Christopher D Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 2227–2237. Association for Computational Linguistics, 2018.

Shiwon Song. The informational value of segment data disaggregated by underlying industry: Evidence from the textual features of business descriptions. *The Accounting Review*, 96(6):361–396, 2021.